

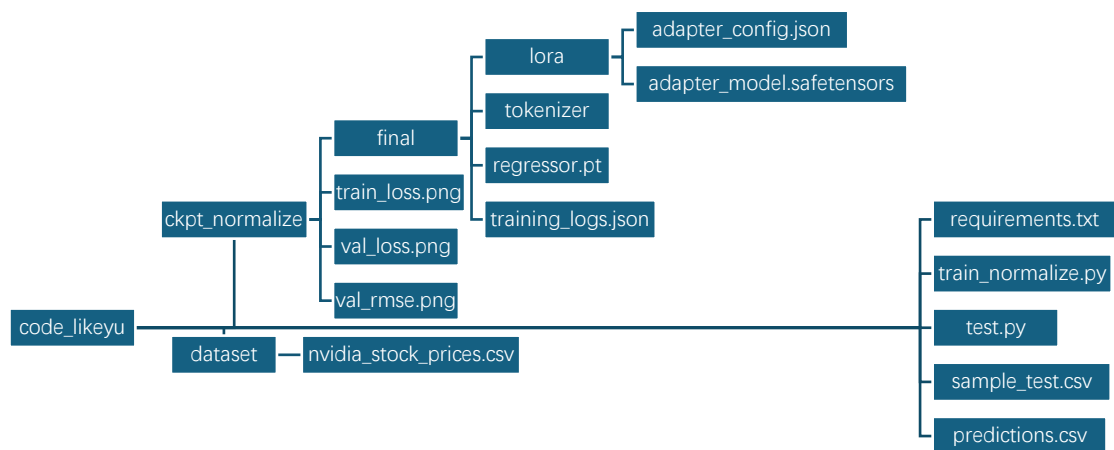
# Code and Report for Stock Price Prediction using Mistral-7B with LoRA Finetuning

LI Keyu

SID: 60060589

2025/11/23

## 1. Code Structure



1. Prepare environment:  
`unzip code_likeyu_final_20251123.zip`  
`cd code_likeyu/`  
`pip install -r requirements.txt`
2. Test the final model:  
`python test.py`
3. (optional) Re-train the LoRA-finetuned Mistral model for stock price prediction:  
`python train_normalize.py`  
The final model will be saved to `ckpt_normalize/final/`

## 2. Dataset

I choose the stock price data of **Nvidia Corporation (NVDA)** from January 2, 2004, to January 1, 2024 as the training data for our method. The source data is download from Kaggle.

- File path: `code_likeyu/dataset/Nvidia_stock_prices.csv`
- Source link: <https://www.kaggle.com/datasets/jvanark/nvidia-daily-stock-price-data>

### 3. LLM Base Model

I choose **Mistral-7B-v0.1** as our base LLM model as its pretrained weight is publicly available, lightweight, and commonly used for sequence-based tasks. It is perfectly supported by PEFT.

## 4. Stock-Mistral-Model Design

### 4.1 LLM Backbone

I choose **Mistral-7B-v0.1** as our base LLM model as its pretrained weight is publicly available, lightweight, and commonly used for sequence-based tasks. It is perfectly supported by PEFT.

### 4.2 LLM-based Regression

As we need to finetune the model for a regression task, i.e., Stock Price Prediction, we came up with two ways to achieve this goal:

- 1) directly using the text output of LLM and turn it into floats,
- 2) adapt the original LLM model into a regression model for “real” float prediction.

**Our choice:** Since the price prediction needs high accuracy, I choose the second method to **modify the original model and add a trainable regression head**. The regression head is designed as a simple, lightweight linear layer. The LLM backbone with LoRA adaptors is used to extract the last hidden state features and the regression head outputs the value for prediction. We use Huber loss to improve the robustness of training.

### 4.3 Handling of Unseen Test Data

For the definition of the model output, I tried two implementations:

- 1) directly output the absolute close value of the next day,
- 2) output the normalized return, i.e., change of close value compared to the last close value.

During experiments, I found that **the first method is very sensitive to the actual scale of data**, which may cause differences between training and testing. For instance, the close value of Nvidia is commonly below 10, while the close value of Apple is commonly bigger than 100.

**Our solution:** In view of this, I defined the output of the model as the **changing rate of close value compared to the last close value of the given time window**, to make the trained model easily adapt to unseen test data. During evaluation, the last close value is provided and we can compute the predicted close value using the model output and the last close value.

## 4.4 Final Model Architecture

The overall model architecture is defined as:

```
class RegressionMistral(torch.nn.Module):
    def __init__(self, model_name):
        super().__init__()
        self.backbone = AutoModel.from_pretrained(model_name, torch_dtype=torch.float16,
device_map="auto")
        self.regressor = torch.nn.Linear(self.backbone.config.hidden_size, 1)

    def forward(self, input_ids, attention_mask, labels=None, last_close=None, return_pred_close=False):
        h = self.backbone(input_ids=input_ids, attention_mask=attention_mask)
        pooled = h.last_hidden_state[:, -1, :]
        logits = self.regressor(pooled).squeeze(-1)      # 预测收益率

        if return_pred_close and last_close is not None:
            # 反归一化
            pred_close = logits * last_close + last_close
        else:
            pred_close = logits

        loss = None
        if labels is not None:
            loss = torch.nn.functional.smooth_l1_loss(logits, labels.float())
        return {"loss": loss, "logits": pred_close}
```

## 5. Fine-tuning Method

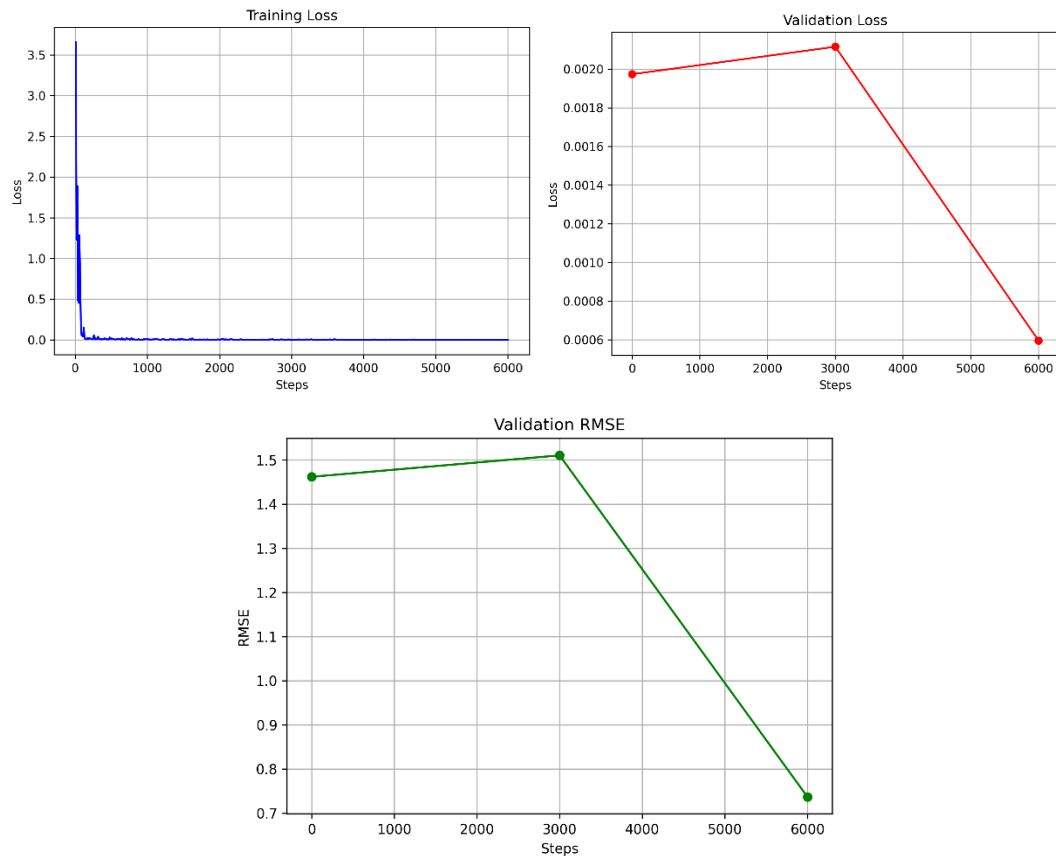
For fine-tuning of the LLM, I chose **LoRA (Low-Rank Adaptation)** as it is a highly-efficient and commonly used technique for fine-tuning LLMs with satisfying performance. This method injects trainable low-rank decomposition matrices into each layer. Instead of updating the full weight matrix, LoRA learns two small matrices whose product represents the weight updates. Typically adds only 0.1-0.5% additional parameters.

### 5.1 Implementation details

- Batch size: 2
- Learning rate: 2e-4
- Total epochs: 3
- Window size: 30

## 6. Learning Curves

The training takes 6003 steps (3 epochs) in total and the training GPU memory usage is about 16 GB. The learning curves using our method in the 3 training epochs are shown below.



## 7. Test Results

We modify the provided framework.py as test.py to implement our finetuned model. The predictions and GPU usage results are printed below:

Total trainable parameters: 6,819,841

Peak GPU memory usage (MB): 13658.65

Final predictions: ['269.25', '269.75', '271.00']

Predictions saved to 'predictions.csv'

'predictions.csv':

|   | A                    | B                    | C                    |
|---|----------------------|----------------------|----------------------|
| 1 | Predicted_Close_Day1 | Predicted_Close_Day2 | Predicted_Close_Day3 |
| 2 | 269.25               | 269.75               | 271                  |